Day 8 questions: wire, reg and reset

HDLBits Attempt 2 | Checked 5 September 2026. Questions were read directly from the current Chrome editors. Entries 127 and 129 show successful simulation results. Entry 123 remains unfinished; its question is answered without marking it complete.

Entry 129: why cannot these wires be reg?

```
wire [15:0] aLower = a[15:0];
// i cant make them reg why
```

This declaration creates a continuously driven connection. Whenever the lower half of a changes, aLower follows it. The equals sign here belongs to a net declaration assignment; it does not mean an assignment executed once at startup.

```
wire [15:0] aLower;
assign aLower = a[15:0];  // same connection
```

Changing only wire to reg changes the meaning of the declaration: a variable declaration initializer is a startup initialization, not a continuously tracking expression. If a changes later, the initializer does not execute again. An initially unknown input can also leave that variable unknown. The signal needs either a net connection or an explicit procedural assignment that runs when the inputs change.

```
reg [15:0] aLower;
always @(*) begin
    aLower = a[15:0];
end
```

This alternative describes combinational logic because every execution assigns aLower and the sensitivity covers the expression inputs. The keyword reg itself does not demand a physical register. In this particular adder, the wire form is simpler: no clock or storage is required. You can also connect the input slice directly to the instance port.

```
add16 low (.a(a[15:0]), .b(b[15:0]), .cin(1'b0),
           .sum(sum[15:0]), .cout(coutL));
```

For example, with a = 32'h1234_ABCD, aLower is ABCD. Changing a to 32'h5678_0001 makes the wire become 0001. A variable initialized from the old input has no such update mechanism.

Source: Intel Quartus: variable declaration initialization

Source: HDLBits: Adder 2

Entry 129: why is carry a wire?

```
wire [16:0] carry; // what it is to be wire
assign carry[0] = cin;
assign cout = carry[16];
```

The carry vector is the wiring between full-adder instances. carry[0] receives the external carry-in. Full adder 0 drives carry[1], full adder 1 drives carry[2], and so on until full adder 15 drives carry[16]. Each stage reads a carry from its neighbor. The parent does not assign these connections inside a clocked always block.

In Verilog, use nets such as wire for connections driven by continuous assignments or child-module outputs. Each bit here has one intended driver. A 16-bit adder needs 17 carry connection points: one input plus the 16 stage outputs. Their widths count connections, not extra storage bits.

```
genvar i;
generate
    for (i=0; i<16; i=i+1) begin : stage
        add1 fa (.a(a[i]), .b(b[i]), .cin(carry[i]),
                 .sum(sum[i]), .cout(carry[i+1]));
    end
endgenerate
```

Your commented-out version uses i < 15. That creates stages 0 through 14 only, leaving bit 15 and carry[16] without the required final stage. A generate loop creates hardware instances during elaboration; it does not wait for 16 clock cycles. A ripple adder has a combinational propagation path through those instances.

For a small trace, consider 011 + 001 with carry-in 0. Bit 0 computes 1+1+0, so sum[0]=0 and carry[1]=1. Bit 1 computes 1+0+1, so sum[1]=0 and carry[2]=1. Bit 2 computes 0+0+1, giving sum[2]=1 and carry[3]=0. The result is 100.

HDLBits already provides add16 in this exercise. Keep your experimental implementation commented out and submit top_module plus add1. Use named ports when reusing an adder: the Fadd exercise declares cout before sum, while this add1 interface declares sum before cout. Positional connections must match the actual declaration exactly.

SystemVerilog permits a single continuous or module-output driver for variables in cases where classic Verilog requires a net. That does not make a declaration initializer continuous, nor does it make mixing procedural and continuous drivers valid. For these Verilog exercises, wire for structural connections is the clearest choice.

Source: Intel Quartus: continuous assignment requires a net in Verilog

Entry 127: why not assign q = 0?

```
// assign q = 4'd0; why i cant do that
```

A continuous assignment drives its target all the time. It is not a one-time reset. Even if q were a wire, this statement would permanently drive zero, so the output could never display loaded data or shift. In the current module, q is also written inside an always block; adding another driver conflicts with that ownership. Classic Verilog additionally requires the continuous-assignment target to be a net, whereas your q is declared reg.

An asynchronous reset is part of the flip-flop behavior: asserting areset clears q immediately, even between clock edges. Deasserting reset lets the existing state hold until the next rising clock edge. At that edge, load has priority over ena. If neither is active, the register retains its previous value.

```
always @(posedge clk or posedge areset) begin
    if (areset)
        q <= 4'b0000;
    else if (load)
        q <= data;
    else if (ena)
        q <= {1'b0, q[3:1]};
end
```

For a trace, load data=1011 at a rising edge: q becomes 1011. At the next enabled rising edge, q becomes 0101; the following enabled edge produces 0010. If ena=0 and load=0, q stays 0010. If reset rises between edges, q immediately becomes 0000. A permanently driven zero could not produce the first three states.

The missing final else intentionally means hold for a clocked register; it does not infer a latch. q <= q is unnecessary here. Nonblocking assignments model the clocked state update, so the right-hand side q[3:1] uses the pre-edge value.

A register initializer is a separate concept from an external reset. Initial values may be supported by particular FPGA flows, but they do not respond when areset changes during operation. Keep the reset logic in the event-controlled block as required by the problem.

Source: HDLBits: asynchronous reset, synchronous load and shift enable

Source: Intel Quartus: continuous assignment target type

Entry 123: do I reset seconds here?

```
if (minutes == 8'd59 && second == 8'd59) begin
    minutes <= 8'd0;
    // will i make seconds 0 here ?
end
```

No second assignment to seconds is needed if your seconds rollover logic already sets second to zero when it reaches 59. At the same enabled rising edge, the minute logic can test the old second value and advance minutes. Nonblocking assignments allow both actions to use the pre-edge values, then update together.

For BCD, 59 is 8'h59 (0101_1001), not 8'd59 (0011_1011). The latter has a ones nibble of B, which is not a valid decimal digit. Your current comparisons mix binary constants with BCD state. Use BCD constants consistently.

```
if (reset) begin
    second  <= 8'h00;
    minutes <= 8'h00;
    hour    <= 8'h12;
    pm      <= 1'b0;
end else if (ena) begin
    // seconds advance every enabled edge
    if (second == 8'h59)
        second <= 8'h00;
    else if (second[3:0] == 4'd9)
        second <= {second[7:4]+4'd1, 4'd0};
    else
        second <= second + 8'd1;

    // minutes advance ONLY at second rollover
    if (second == 8'h59) begin
        if (minutes == 8'h59)
            minutes <= 8'h00;
        else if (minutes[3:0] == 4'd9)
            minutes <= {minutes[7:4]+4'd1, 4'd0};
        else
            minutes <= minutes + 8'd1;
    end
    // hour/pm logic also goes here, gated by
    // second == 8'h59 && minutes == 8'h59
end
```

This is the seconds/minutes pattern, not a complete submitted solution. For hours: at minute-and-second rollover, 11 becomes 12 and toggles pm; 12 becomes 01 without toggling pm; 09 becomes 10. Other hour values increment their ones digit. Use pm <= ~pm so both AM-to-PM and PM-to-AM transitions work.

Also fix the current minutes[7:5] slice to [7:4], clear the ones digit when a BCD digit carries, reset all time registers, and declare pm as output reg if it is procedurally assigned. The current page reports Incorrect, so this entry stays Pending.

Source: HDLBits: 12-hour BCD clock specification